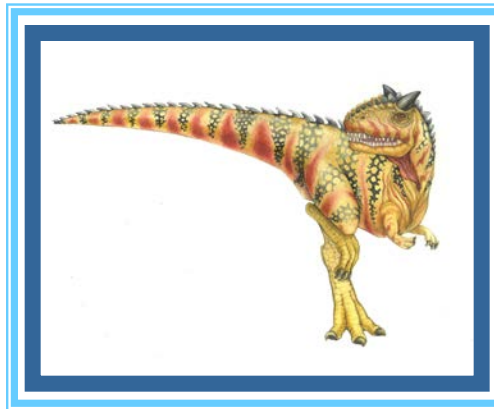


Chapter 7: Deadlocks





Chapter 7: Deadlocks

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock





Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks
- To present a number of different methods for preventing or avoiding deadlocks in a computer system





System Model

System consists of resources:

A computer system has different types of resources needed by processes to run.

Resource types $R_1, R_2, \dots, R_m$:

There are m different kinds of resources, for example: CPU cycles, memory space, I/O devices.

Each resource type R_i has W_i instances:

Each resource type has a certain number of units. For example, there may be 4 CPUs, 2 printers, 8 GB of RAM, etc.

Each process utilizes a resource as follows:

Request: The process asks for the resources it needs.

Use: The process uses the allocated resources to perform its task.

Release: The process releases the resources when it no longer needs them, making them available for other processes.

This basic model is used in operating systems to manage resources safely and efficiently.





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** A resource (like a printer or file) can only be used by one process at a time.
- **Hold and wait:** A process is using at least one resource and is also waiting to get more resources that are being used by other processes.
- **No preemption:** A process cannot be forced to give up a resource; it must release it on its own when it's finished using it.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .





Deadlock with Mutex Locks

- Deadlocks can occur via system calls, locking, etc.
- See example box in text page 318 for mutex deadlock





Resource-Allocation Graph

A Resource Allocation Graph (RAG) is a way to visually show how resources and processes are related in a system. It helps you understand and detect deadlocks. A Resource Allocation Graph has:

1. A set of nodes (V) — these are the points in the graph.
There are two kinds of nodes:

Processes (P): These are all the running programs. Example: $P_1, P_2, \dots, P_n$.

Resources (R): These are all the resource types. Example: $R_1, R_2, \dots, R_m$.

2. A set of edges (E) — these are the arrows that show how processes and resources are connected.
There are two kinds of arrows:

Request edge ($P \rightarrow R$): An arrow from a process to a resource means the process is asking for that resource.

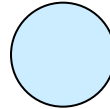
Assignment edge ($R \rightarrow P$): An arrow from a resource to a process means that resource has been given to that process.





Resource-Allocation Graph (Cont.)

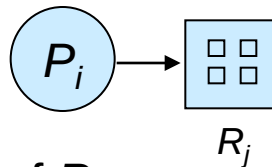
- Process



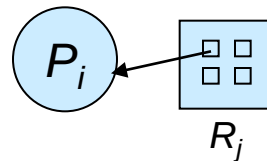
- Resource Type with 4 instances



- P_i requests instance of R_j

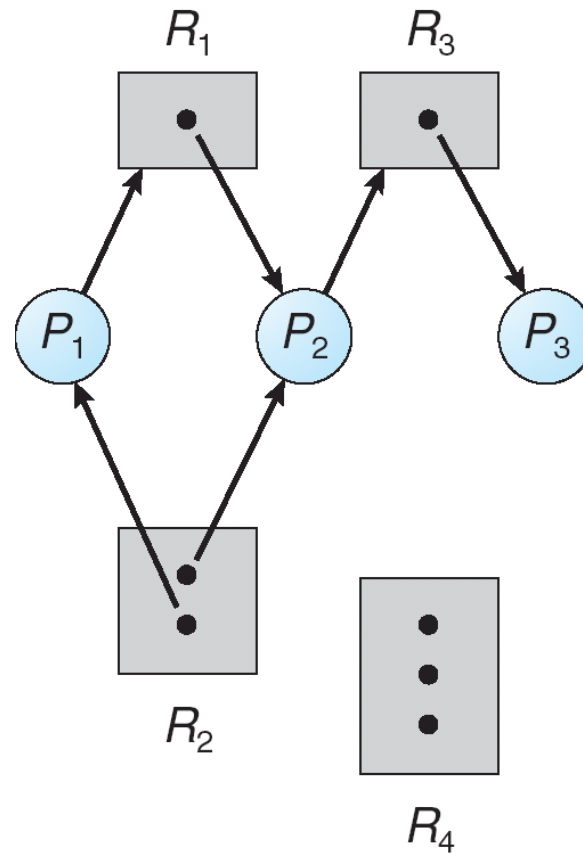


- P_i is holding an instance of R_j



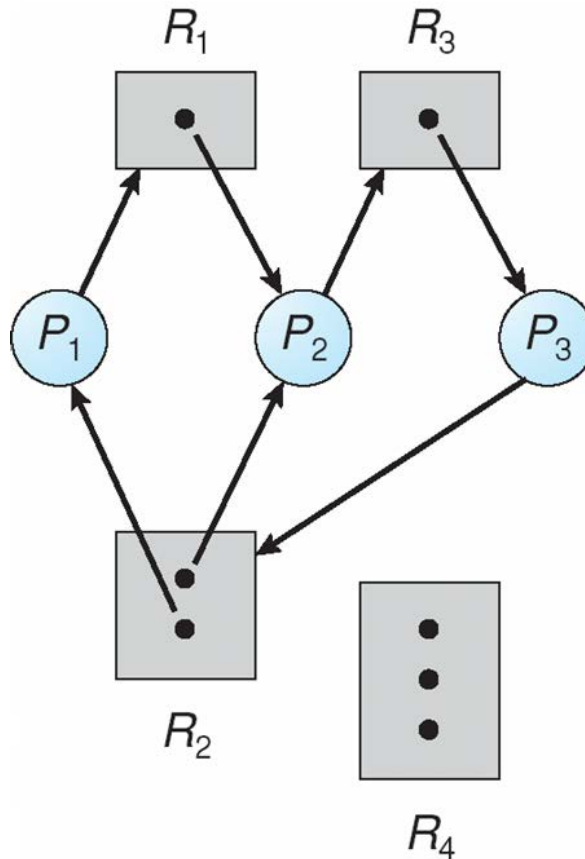


Example of a Resource Allocation Graph



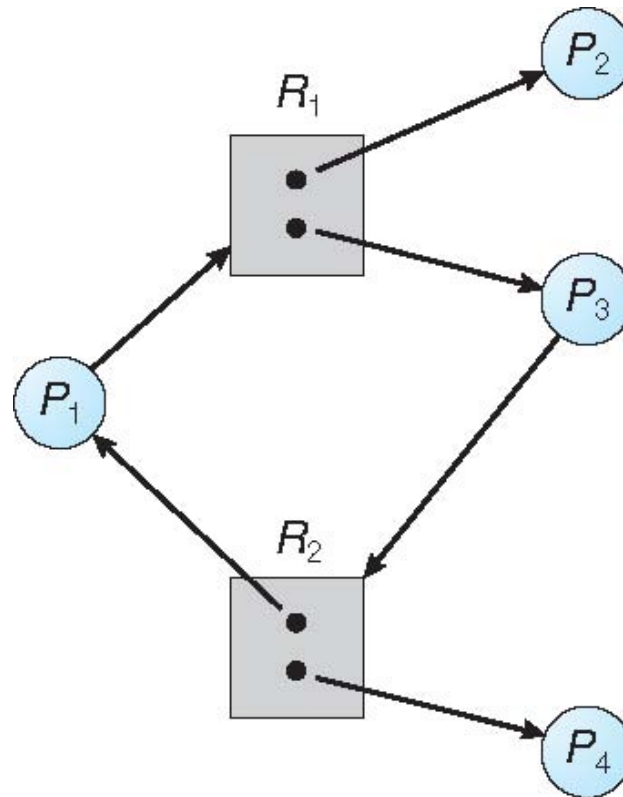


Resource Allocation Graph With A Deadlock





Graph With A Cycle But No Deadlock





Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - If each resource type has only one unit, then there is a deadlock for sure
 - If resources have more than one unit, there might be a deadlock, but not always — it depends on whether the waiting can be resolved with the free units.





Methods for Handling Deadlocks

Methods for Handling Deadlocks

Make sure deadlocks never happen:

- 1. Deadlock Prevention:** Deadlock Prevention is a technique in operating systems that ensures deadlock never occurs by breaking at least one of the four necessary conditions of deadlock.
- 2. Deadlock Avoidance:** Deadlock Avoidance is a technique where the operating system carefully allocates resources so that deadlock can never happen, even if all four necessary conditions exist.
- 3. Let deadlocks happen, then fix them:** Allow deadlocks to occur, then detect them and take steps to break the deadlock and recover.
- 4. Ignore the problem:** Do nothing about deadlocks. Just assume they won't happen. (This is what most operating systems do, like UNIX.)





Deadlocks

Deadlock can only occur when all four of these conditions hold at the same time. If at least one condition is prevented or broken, the system can avoid deadlock situations.

Mutual Exclusion Prevention

Make resources shareable whenever possible. If multiple processes can use a resource at the same time, deadlock cannot happen. Example: Read-only files can be shared; printers usually cannot.

Hold and Wait Prevention

Make sure processes do not hold resources while waiting for others. This can be done by requiring processes to request all resources at once or forcing them to release held resources before requesting new ones.

No Preemption Prevention

Allow the system to take resources away from processes if needed. If a process is waiting, resources it already holds can be taken and given to others. This prevents processes from getting stuck.

Circular Wait Prevention

Give all resources a fixed order. Processes must request resources following this order. This stops cycles from forming, so circular waiting cannot happen.





Deadlock Prevention

Restrain the ways request can be made

- **Mutual Exclusion:**

- This means some resources cannot be shared — for example, a printer.
- But some resources can be shared — for example, read-only files — so mutual exclusion is not needed for them.

- **Hold and Wait:**

- To prevent deadlock, don't allow a process to hold some resources while waiting for others.
- This can be done in two ways:
 1. Make the process ask for **all the resources it will need** at once, before it starts running.
 2. Or, only let the process ask for new resources **when it has none at all** already.
- **Problems with this approach:**
 - This can lead to **low resource use** (resources may stay unused because processes grab more than they need just in case).
 - Some processes may **starve** (never get resources) because others are holding too many at once.





Deadlock Prevention (Cont.)

No Preemption

- If a process is using some resources and asks for another one that isn't available **right now**, it must **give up** all the resources it's already holding.
- The resources it gave up go back into the pool, and the process must wait for **all** the resources it needs — the old ones and the new one — to become available again.
- The process will **restart** only when it can get **everything it needs at once**.

Circular Wait

- Deadlocks can happen when processes hold resources in a **circle**, each waiting for the next.
- To stop this, put **all resources in an order** (like $R1 < R2 < R3$).
- Make processes **always request resources in that order** — for example, they can't ask for R1 after they already hold R2.
- This breaks the circle because it's impossible to go backward in the order.





Deadlock Example

```
/* thread one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /** * Do some work */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);
}

/* thread two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /** * Do some work */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);
}
```





Deadlock Example with Lock Ordering

```
void transaction(Account from, Account to, double amount)
{
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);
    acquire(lock1);
    acquire(lock2);
    withdraw(from, amount);
    deposit(to, amount);
    release(lock2);
    release(lock1);
}
```

Transactions 1 and 2 execute concurrently. Transaction 1 transfers \$25 from account A to account B, and Transaction 2 transfers \$50 from account B to account A





Deadlock Avoidance

Deadlock Avoidance

1. To avoid deadlock, the system needs some extra information in advance.
2. The simplest way is: each process must say the maximum number of resources of each type it might need.
3. The deadlock avoidance algorithm keeps checking the resource situation to make sure there is no circular waiting.
4. The resource situation means how many resources are free, how many are given to processes, and how many resources each process might ask for at most.





Safe State

A system is in a safe state if there exists at least one sequence of all processes such that each process can get its required resources, run to completion, and release its resources without causing deadlock.

If a process P_i can't get what it needs right now, it can wait for others to finish. When one process finishes, it gives back its resources, so the waiting process can get what it needs, run, and finish.

Then the next process in the order can do the same, and so on.





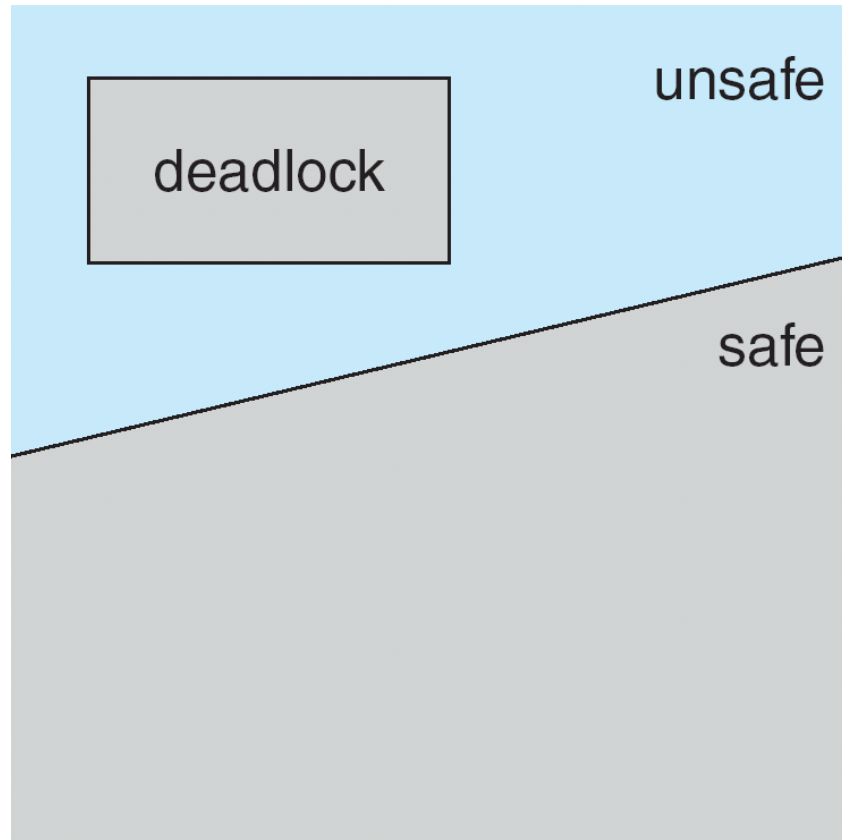
Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.





Safe, Unsafe, Deadlock State





Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm





Resource-Allocation Graph Scheme

Resource-Allocation Graph Scheme

A claim edge ($P \rightarrow R$) means process P might ask for resource R in the future. This is shown with a dashed line.

When the process actually asks for the resource, the claim edge changes to a request edge.

When the resource is given to the process, the request edge changes to an assignment edge.

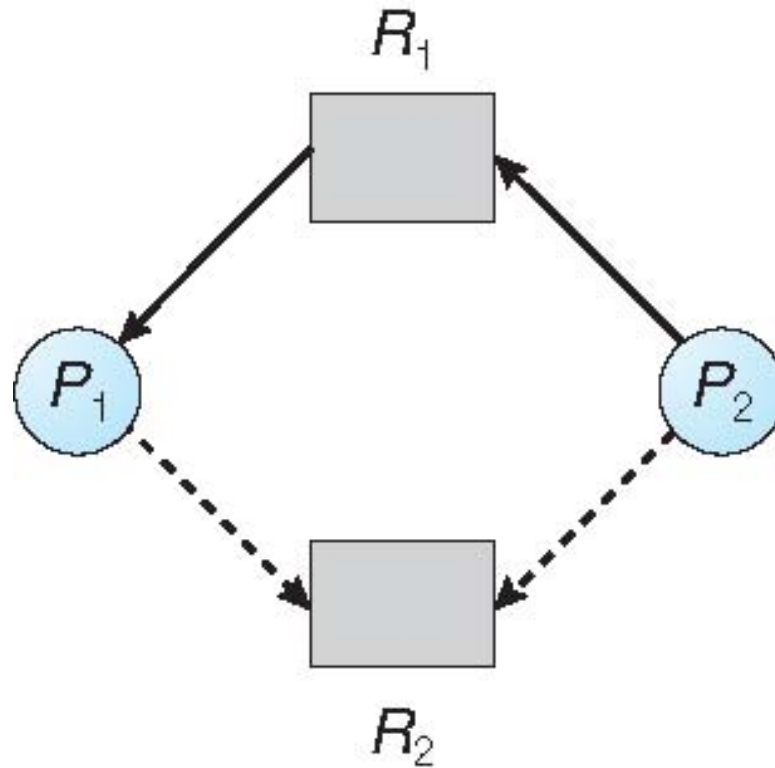
When the process is done and releases the resource, the assignment edge changes back to a claim edge.

Every resource that a process might need must be told to the system ahead of time.



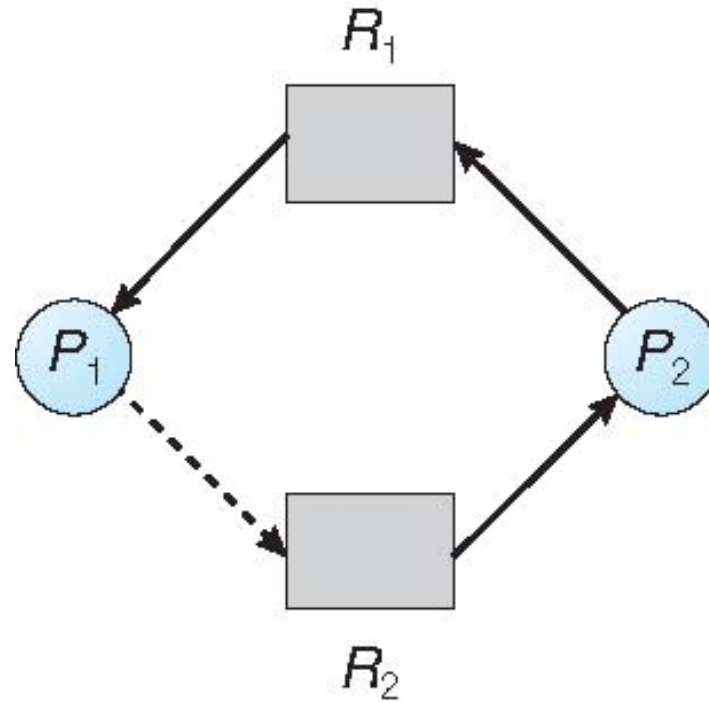


Resource-Allocation Graph





Unsafe State In Resource-Allocation Graph





Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph





Banker's Algorithm

- Multiple instances of each resource can exist.
- Each process must declare in advance the maximum number of resources it may need.
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time





Banker's Algorithm

The Banker's Algorithm is a deadlock avoidance method used in operating systems to allocate resources safely. It ensures the system never enters an unsafe state. It is called "Banker's Algorithm" because it works like a bank lending money: it only gives out resources if it can guarantee that all processes will finish eventually.

How it Works:

1. Multiple instances of resources are allowed.
2. Maximum claim: Each process must declare the maximum number of each resource it may need before execution.
3. Safe allocation: When a process requests resources, the system checks if granting the request keeps the system in a safe state.
 - If yes → request is granted.
 - If no → process **must wait** until resources are available.
4. Resource release: Once a process finishes using its resources, it returns them to the system so other processes can use them.





Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If $available[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$





Safety Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively.
Initialize:

Work = Available

Finish [i] = **false** for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) **Finish** [i] = **false**

(b) **Need** _{i} ≤ **Work**

If no such i exists, go to step 4

3. **Work = Work + Allocation** _{i}
Finish [i] = **true**
go to step 2

4. If **Finish** [i] == **true** for all i , then the system is in a safe state





Resource-Request Algorithm for Process P_i

$Request_i$ = request vector for process P_i . If **$Request_i[j] = k$** then process P_i wants k instances of resource type R_j

1. If **$Request_i \leq Need_i$** , go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If **$Request_i \leq Available$** , go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored





Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	





Example (Cont.)

- The content of the matrix **Need** is defined to be **Max – Allocation**

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria





Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?





Deadlock Detection

Deadlock Detection is a method where the operating system allows deadlocks to occur, but has a mechanism to detect them and recover. Unlike prevention or avoidance, the system does not try to stop deadlocks in advance.

- Allow system to enter deadlock state
- Run detection algorithm
- Take Recovery Action
 1. Terminate processes: Kill one or more processes involved in the deadlock.
 2. Preempt resources: Take resources from some processes and give them to others to break the deadlock.





Single Instance of Each Resource Type

A wait-for graph is a simple way to show which processes are waiting for other processes to release resources.

Single Instance of Each Resource Type

1. Use a wait-for graph to keep track of which process is waiting for which other process.

2. In this graph:

The nodes are the processes.

An arrow from P_i to P_j means Process P_i is waiting for Process P_j to release a resource.

3. From time to time, run an algorithm to check if there is a cycle in this graph.

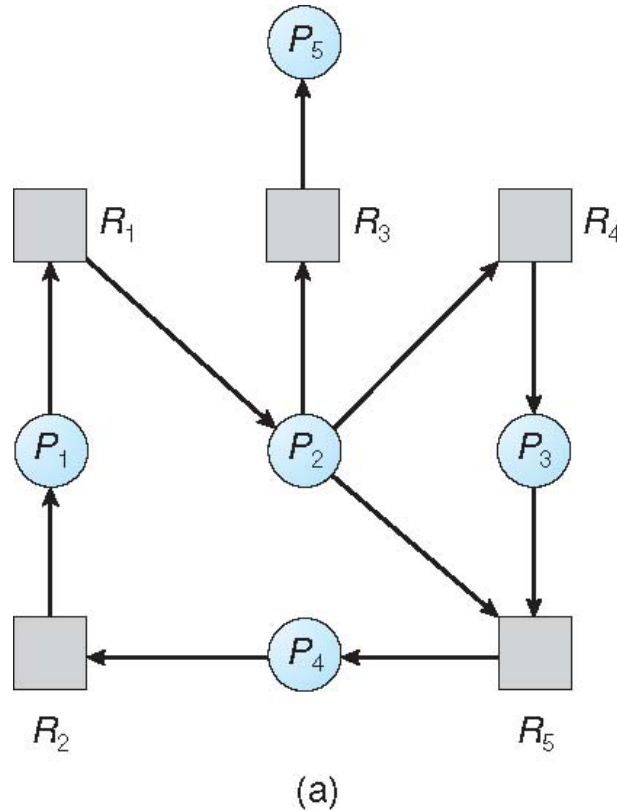
If there is a cycle, then a deadlock exists.

4. To find a cycle in the graph, the algorithm needs about n^2 operations, where n is the number of processes (nodes).

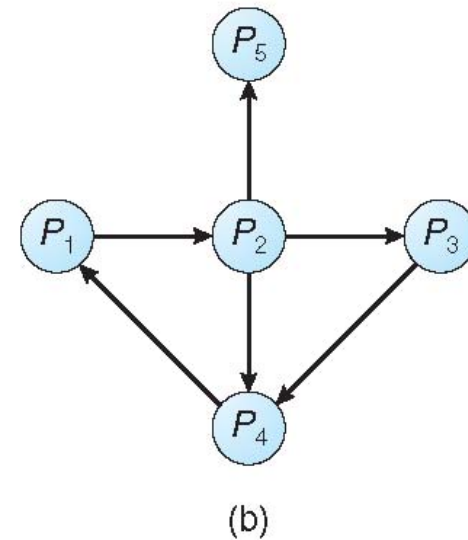




Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph





Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process
- **Request:** An $n \times m$ matrix indicates the current request of each process. If **Request** $[i][j] = k$, then process P_i is requesting k more instances of resource type R_j .





Detection Algorithm

1. Let **Work** and **Finish** be vectors of length **m** and **n**, respectively
Initialize:
 - (a) **Work = Available**
 - (b) For $i = 1, 2, \dots, n$, if **Allocation_i $\neq 0$** , then
Finish[i] = false; otherwise, **Finish[i] = true**
2. Find an index **i** such that both:
 - (a) **Finish[i] == false**
 - (b) **Request_i $\leq$ Work**

If no such **i** exists, go to step 4





Detection Algorithm (Cont.)

3. **$Work = Work + Allocation_i$**
 $Finish[i] = true$
go to step 2
4. If **$Finish[i] == false$** , for some i , $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if **$Finish[i] == false$** , then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state





Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in ***Finish[i] = true*** for all i





Example (Cont.)

- P_2 requests an additional instance of type **C**

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4





Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - ▶ one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.





Recovery from Deadlock: Process Termination

- One way to get out of a deadlock is to terminate (abort) processes.
- You can either:
 1. Abort all deadlocked processes at once, or
 2. Abort one process at a time until the deadlock is gone.
- In which order should we choose to abort?
 1. The priority of the process (important processes might be kept).
 2. How long the process has run so far and how much longer it needs to finish.
 3. How many resources the process has already used.
 4. How many more resources it still needs to finish.
 5. How many total processes would need to be killed to break the deadlock.
 6. Whether the process is interactive (user-facing) or batch (background).





Recovery from Deadlock: Resource Preemption

Another method is resource preemption, which involves taking resources away from certain processes.

Select a victim: A process is chosen to lose its resource, aiming to minimize the overall cost.

Rollback: When a resource is taken back, the affected process may need to roll back (undo its work) and restart from a safe point.

Starvation: If the same process is always selected as the victim, it may never complete its task. To avoid this, the system keeps track of how many times each process has been rolled back and tries not to pick the same one repeatedly.



End of Chapter 7

